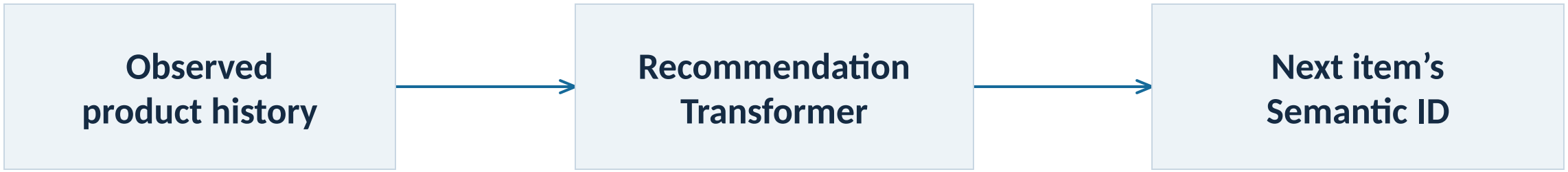
Generative retrieval for next-item recommendation

TIGER-inspired teaching implementation



Frozen Sentence-T5 embeddings feed the Semantic ID quantizer

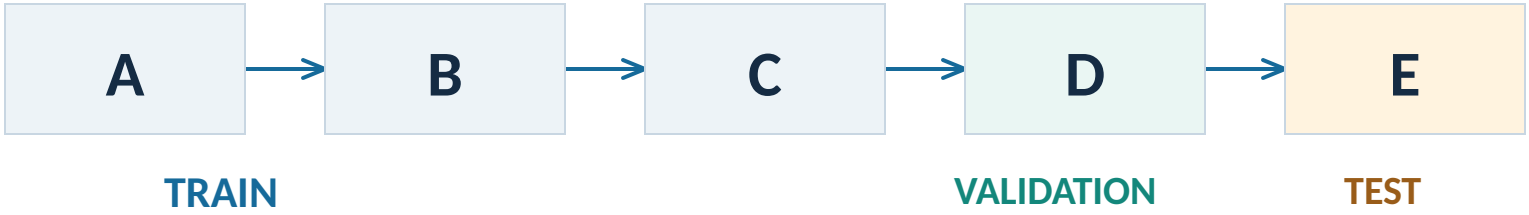
One real-data CPU experiment, compared with two simple baselines

README.md; configs/poc01.yaml

Data and chronological holdout

Amazon Beauty 2014 2,000 users 7,154 items

Time order



Training: [A] → B and [A,B] → C
Validation: [A,B,C] → D
Test: [A,B,C,D] → E

17,264 events → 13,264 train events → 11,264 examples

D is observed at test time. D and E are excluded from that user's training targets.

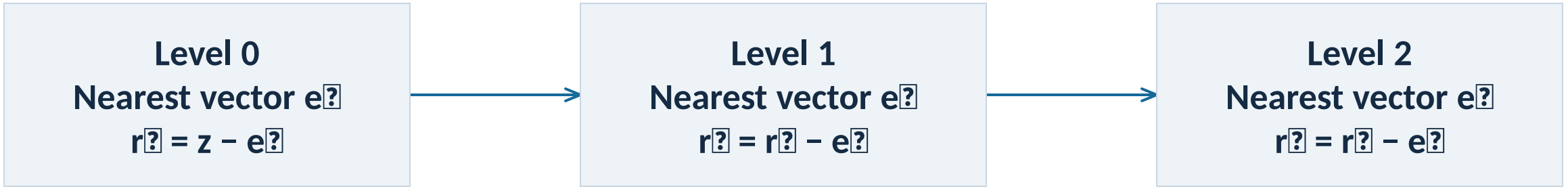
src/data.py: make_subset; src/dataset.py: build_train_examples

Product content becomes unique Semantic IDs

Title, brand, category, price → frozen Sentence-T5 [768]

Encoder: 768 → 512 → 256 → 128 → 32

Each level selects from 256 vectors



Reconstruct the embedding: decoder(e₀ + e₁ + e₂)

ID = [c₀ c₁ c₂ collision suffix] 7,154 unique IDs, then frozen

Three learned levels. The fourth code only disambiguates collisions.

src/rqvae.py; src/semantic_ids.py; reports/poc01/id_diagnostics.json

Constructing model inputs and labels

Position	0	1	2	Suffix
Raw ID	236	227	111	0
Add offset	2	258	514	770
Model token	238	485	625	770

Input: [user bucket] + [latest 20 × four-token item IDs] + [EOS]

Input □ 82tokens Target =5 tokens Vocabulary =3,026

PAD = 0 EOS = 1 Label padding = −100 (ignored by the loss)

src/tokenization.py: HistoryEncoder; src/dataset.py: collate

Training and selecting the recommender

Small T5, random initialization, teacher forcing

Step	1	2	3	4	5
Decoder sees	PAD	c□	c□	c□	c□
Predicts label	c□	c□	c□	c□	EOS

AdamW + mean token cross-entropy Budget: 5,000 steps

Validate every 500 steps on 500 users. Best NDCG@10: step 4,500.

src/recommender.py; src/training.py; reports/poc01/experiment.json

Disclosed changes from the paper

Setting	Paper	CPU proof of concept
Encoder + decoder layers	4 + 4	2 + 2
Attention heads × head size	6 × 64	4 × 32
Feed-forward / batch size	1,024 / 256	512 / 64
Beauty training steps	200,000	5,000 + smaller subset

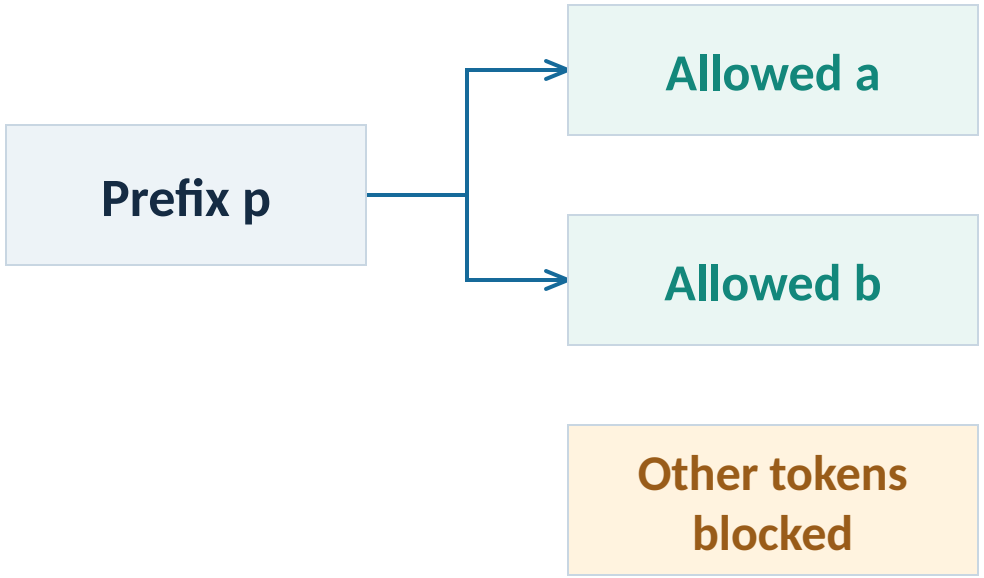
Quantizer: Adam + warm-up
Retrieval: catalog prefix constraints

README.md: settings table; docs/deviations.md; paper §4

Generating and ranking catalog items

Beam width 20 **Four ID tokens + EOS** **Return ten unique items**

Trie example (schematic)



Saved top-ranked output

Generated tokens
[215,310,561,770,1]

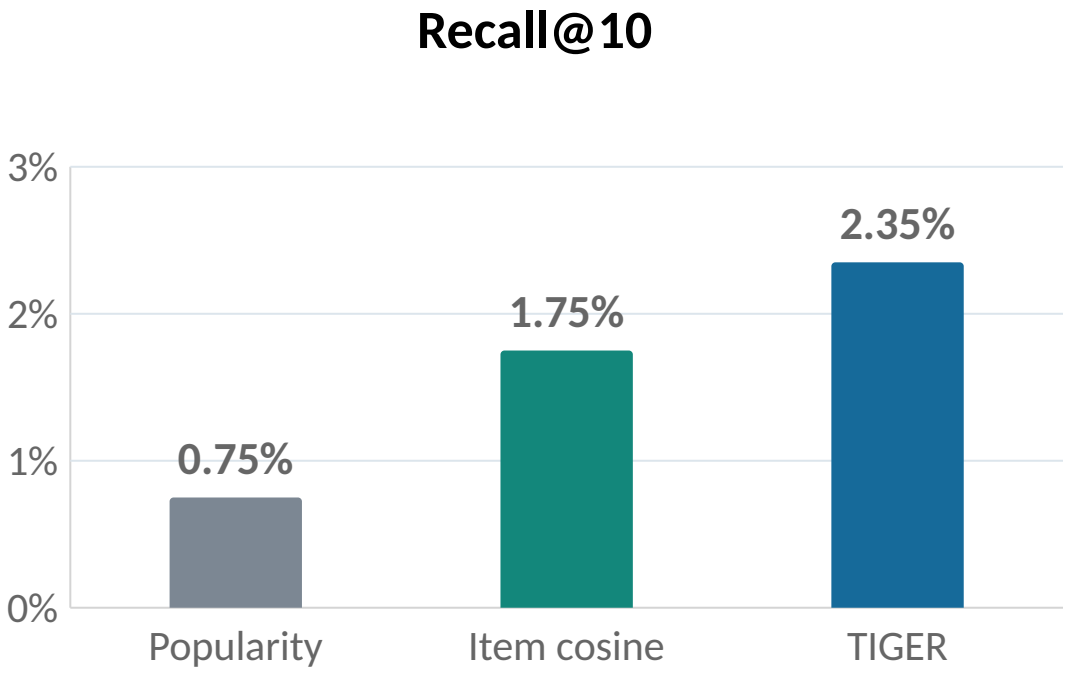
Raw ID [213,52,47,0]
Olay cleansing system

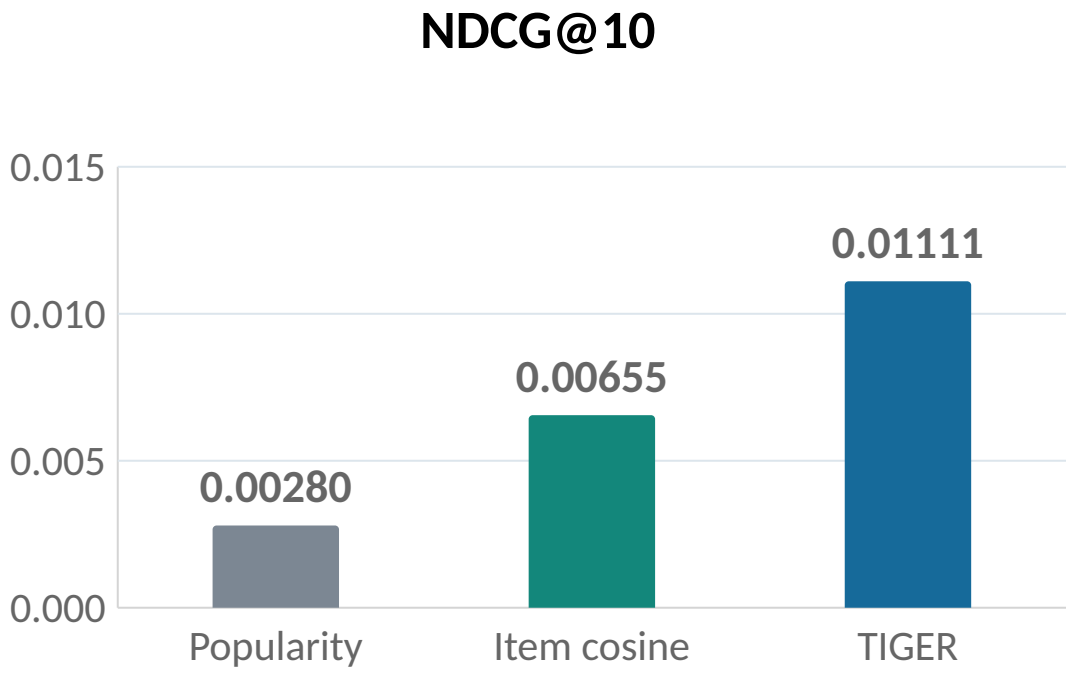
Held-out target: Pureology shampoo

Rank by summed token log-probabilities. Catalog validity does not ensure relevance.

src/retrieval.py: CatalogTrie, Retriever; reports/poc01/example_trace.json

Two baselines and held-out results





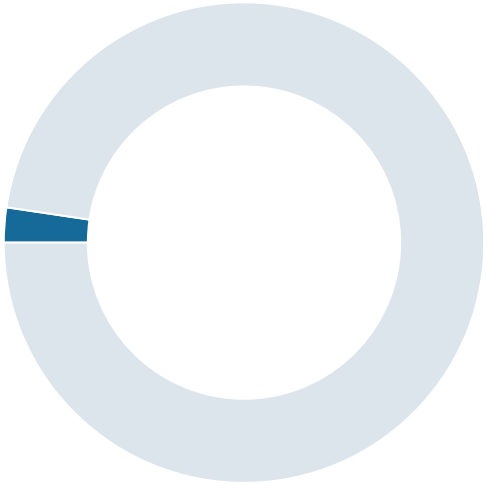
Top-ten hits: 15 / 35 / 47 out of the same 2,000 users

Full catalog, previously seen items allowed. TIGER's frozen scores were not rerun.

src/baselines.py; src/metrics.py; reports/baselines/results_test.json

What the results support and leave open

Top-ten outcomes



Unanswered question	Evidence boundary
Why does it improve?	No random-ID ablation
Does it generalize?	One seed, no significance test
Does it handle unseen items?	36.4% training-unseen targets; subgroup accuracy unknown

47 hits / 1,953 misses

Higher scores than two simple baselines, but 1,953 of 2,000 targets are missed.

reports/presentation_data_profile.json; reports/baselines/results_test.json

Actual contributions and shared preparation

6734446323	Initialization and Developer A code
6734453723	Developer B code
6634424423	Code review and dataset cleanup

Every member prepares to explain all modules.

docs/contributions.md

Backup: quantization and gradient routing

qST = z + (quantized − z).detach()



Forward value = quantized Surrogate gradient □ qST/□=z□

Loss term	Trainable path	Detached side
Reconstruction	Encoder + decoder via STE	Discrete choice / codebook path
Codebook	Selected code vectors	Residual
Commitment	Encoder via residual	Selected code vectors

src/rqvae.py: quantize, forward; tests/test_quantizer.py

Backup: attention and tensor shapes



Input IDs + padding mask
[batch, L], L ≤ 82

Cross-attention
K, V from history
Q from decoder

Labels [batch, 5]
Logits [batch, 5, 3026]

Attention weights use Q–K compatibility to combine values V

Padding masks exclude empty positions. Causal masks exclude future target positions.

src/recommender.py: build_model; src/dataset.py: collate

Backup: item-cosine worked example

Binary user–item incidence

Training user	a	b	c
U1	1	1	0
U2	1	0	1

n(a) = 2 n(b) = 1
shared(a,b) = 1

sim(a,b) = 1 / √(2 × 1)
= 1 / √2
sim(b,c) = 0

Candidate score = sum of similarities to observed history

Binary counts ignore repeated events. Self-similarity = 0. Ties use training popularity.

src/baselines.py: ItemCosine; tests/test_item_cosine.py

Backup: ranking scores and evaluation

Beam score = □ log P(token | history, prefix)

Toy valid path	First-token P	Continuation P	Complete-path P
A	0.60	0.10	0.060
B	0.40	0.90	0.360

Illustration: beam width 1 keeps A and loses the better completed path B

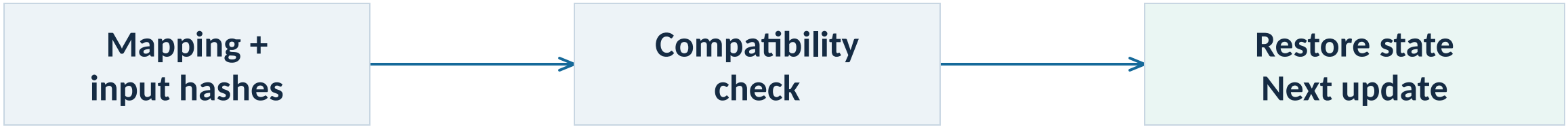
Target at rank 3: Recall@5 = 1 NDCG@5 = 1 / log□ (4) ≈ 0.5

Empty predictions remain in averages. Remove duplicate recommendations.

tests/test_model_retrieval.py; src/metrics.py

Backup: frozen checkpoints and recovery

Saved artifact	What it preserves	Use
best/	Validation-selected inference weights	Frozen evaluation
latest.pt	Model + optimizer + RNG Epoch + batch cursor	Exact CPU continuation



59 passing tests reported. The original experiment recorded 57.

src/training_state.py; tests/test_training_resume.py